

FULL STACK DEVELOPMENT

REACT – O PRINCÍPIO

AULA 05

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	4
SAIBA MAIS	5
O QUE VOCÊ VIU NESTA AULA?	10
REFERÊNCIAS	11

EMANSP

O QUE VEM POR AÍ?

Nesta aula, mergulharemos no emocionante mundo dos React Hooks. Eles revolucionaram a forma como desenvolvemos componentes em aplicações React, oferecendo uma abordagem mais simples e eficiente para gerenciar estado e efeitos colaterais em componentes de função.



HANDS ON

Neste vídeo, vamos mergulhar no mundo dos React Hooks. Os Hooks são uma característica poderosa do React que nos permite adicionar funcionalidades de estado e ciclo de vida aos componentes de função. Vamos explorar como usar Hooks para criar componentes mais eficientes e reutilizáveis em nosso projeto.



SAIBA MAIS

Introdução aos React Hooks

Os React Hooks são funções especiais que permitem que você use o estado e outros recursos do React sem escrever uma classe. Eles foram introduzidos na versão 16.8 do React e têm sido amplamente utilizados desde então. Existem diversos tipos de Hooks, como `useState`, `useEffect`, `useContext`, entre outros, que nos ajudam a resolver diferentes problemas em nossos componentes de função.

Vantagens

- Permitem o uso de estado e outras funcionalidades do React em componentes de função;
- Reduzem a duplicação de código e tornam os componentes mais legíveis;
- Simplificam a lógica de componentes complexos, facilitando a manutenção.

Desvantagens

- Podem aumentar a complexidade de componentes em projetos grandes se não forem utilizados com moderação e organização adequada;
- Algumas práticas comuns de uso de classes, como o ciclo de vida de montagem, podem parecer diferentes ao usar Hooks, o que pode exigir um período de adaptação.

Iniciando o projeto React com Vite

Aproveitando o outro projeto utilizado na aula de Styled components, adicionaremos um componente de contador. Vamos começar explorando alguns Hooks padrão do React, como `useState` e `useEffect`.

useState: este Hook nos permite adicionar estado a componentes de função. Veremos como usar `useState` para criar um contador simples e controlar seu estado.

useEffect: o Hook `useEffect` nos permite executar efeitos colaterais em componentes de função. Vamos aprender a usá-lo para buscar dados de uma API externa e atualizar o estado do componente.

Criando um componente com React Hooks

Agora, vamos criar um componente de exemplo usando React Hooks. Criaremos um componente de contador que utiliza useState para controlar seu estado interno.

1. No diretório components, crie um novo arquivo chamado Counter.js.
2. Dentro do arquivo Counter.js, importe useState do React e crie uma função de componente de função chamada Counter.
3. Dentro da função Counter, use o useState para inicializar uma variável de estado para o contador e uma função para atualizá-la. Renderize o valor do contador e adicione botões para incrementar e decrementar o contador.

Testando o componente

Agora, vamos importar e usar nosso componente Counter em nosso aplicativo principal.

1. No arquivo App.js, dentro do diretório src, importe o componente Counter.
2. Renderize o componente Counter dentro do componente App.

[Acesse o projeto!](#)

Estrutura do diretório

Para facilitar o entendimento, temos uma estrutura parecida com a da figura 1.

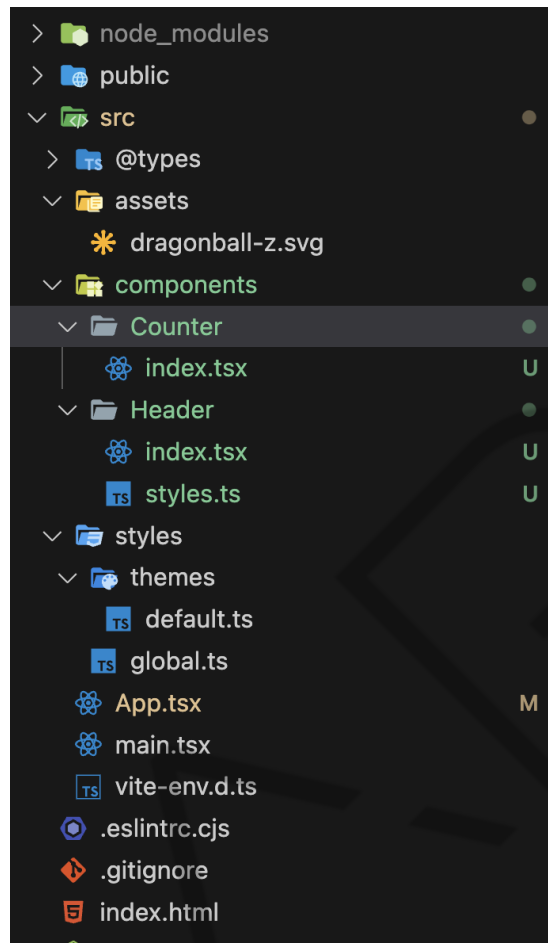


Figura 1 – Estrutura
Fonte: elaborada pela autora (2024)

Vejamos mais um pouco sobre os Hooks `useState`, `useEffect`, `useContext`, `useReducer`, `useCallback` e `useMemo`.

useState: permite adicionar estado a componentes de função. Com `useState`, você pode declarar variáveis de estado e funções para atualizá-las. Por exemplo:

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>O valor atual do contador é: {count}</p>
      <button onClick={() => setCount(count + 1)}>Incrementar</button>
    </div>
  );
}
```

useEffect: permite executar efeitos colaterais em componentes de função, como buscar dados de uma API externa, manipular o DOM ou configurar eventos. Por exemplo:

```
import React, { useState, useEffect } from 'react';

function Example() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Você clicou ${count} vezes`;
  }, [count]);

  return (
    <div>
      <p>Você clicou {count} vezes</p>
      <button onClick={() => setCount(count + 1)}>Clique aqui</button>
    </div>
  );
}
```

useContext: permite acessar um contexto fornecido pelo Context API do React. Isso é útil para compartilhar dados entre componentes sem a necessidade de passá-los manualmente por meio de props.

useReducer: permite gerenciar o estado de um componente usando o padrão de design reducer, semelhante ao que é usado em Redux.

useCallback e **useMemo:** esses Hooks são úteis para otimizar o desempenho, memorizando funções ou valores calculados para evitar cálculos desnecessários em re-renderizações.

Esses são apenas alguns exemplos dos tipos de React Hooks disponíveis. Eles são extremamente versáteis e podem ser combinados e personalizados para atender às necessidades específicas do seu aplicativo. Com os React Hooks, é possível escrever componentes de função mais limpos, concisos e reutilizáveis em suas aplicações React.

Ambiente de desenvolvimento

- [Visual Studio Code](#);
- [Node](#), de preferência a versão LTS (obrigatório para evitar problemas com pacotes);
- [Typescript](#) (instalar em ambiente global);
- [Vite](#), builder do projeto.

EXEMPLO

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos os fundamentos dos React Hooks e como usá-los para adicionar funcionalidades de estado e ciclo de vida aos componentes de função em nosso projeto React.

EMSE

REFERÊNCIAS

ADRIANO, T. **Typescript**. [s.l.]: Casa do Código. 2021.

BABEL. **Babel**. [s.d.]. Disponível em: <https://babeljs.io/>. Acesso em: 28 jun. 2024.

CARVALHO, T. **React: o que é e como funciona essa ferramenta?** Disponível em: <https://medium.com/reactbrasil/react-o-que-%C3%A9-e-como-funciona-essa-ferramenta-319922a8371c>. Acesso em: 28 jun. 2024.

REACT. **Documentação React**. [s.d.]. Disponível em: <https://react.dev/>. Acesso em: 28 jun. 2024.

STYLE COMPONENTS. **Styled Components**. [s.d.]. Disponível em: <https://styled-components.com/>. Acesso em: 28 jun 2024.

PALAVRAS-CHAVE

Palavras-chave: ReactJs. Front-end. Biblioteca. React-hooks.

EMSE

POS TECH